

Robust Adaptive Control for Vehicle Lateral Dynamics During Emergency Maneuvers

Yufei Xi and Jyotshna Dammu
 {xiyufei, jyotshna}@umich.edu

I. INTRODUCTION

Advanced driver assistance systems (ADAS) and automated driving technologies rely heavily on accurate modeling and control of vehicle lateral dynamics to ensure safety, stability and passenger comfort. Functions such as lane keeping, lane following and collision avoidance demand precise control of yaw rate and lateral position under a wide range of operating conditions. However, real-world driving environments introduce significant uncertainty due to variations in road geometry and surface conditions, including road curvature, bank angle and tire-road friction.

Traditional vehicle control systems are typically designed using fixed-parameter models and fixed-gain controllers that assume nominal road and tire conditions. While these approaches can perform adequately in steady-state or mildly varying scenarios, their performance degrades when the vehicle encounters abrupt changes in road parameters or executes aggressive maneuvers. This limitation becomes particularly critical during collision-imminent situations, where rapid lateral maneuvers are required, and modeling inaccuracies can directly compromise vehicle stability and safety.

II. PROBLEM STATEMENT

Modern automated driving and ADAS functionalities demand reliable lateral control performance across a wide range of operating conditions, including nominal cruising and highly aggressive safety-critical maneuvers. While classical control approaches assume well-known and fixed vehicle and road parameters, real-world driving environments violate these assumptions due to abrupt changes in road geometry and tire-road interaction. Such uncertainties become especially pronounced during collision-avoidance scenarios, where rapid lateral accelerations amplify modeling errors and can compromise stability and tracking accuracy. These limitations highlight a fundamental gap between nominal controller design and real-world vehicle operation, motivating the exploration of adaptive control frameworks capable of handling unknown and time-varying dynamics in real time.

III. DESIGN MODEL

The controller design is based on a bicycle model that captures the essential lateral and yaw dynamics of the vehicle while maintaining analytical tractability [1]. The vehicle is represented by a single front and rear wheel, with lateral tire forces modeled using a linear tire approximation. This

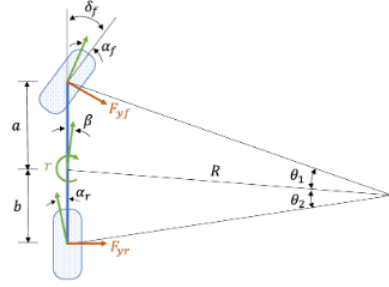


Fig. 1. Sketch of vehicle model.

$$\frac{d}{dt} \begin{bmatrix} y \\ \psi - \psi_d \\ r \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{mu} & 0 \\ 0 & 0 & 1 \\ 0 & \frac{bC_{ar} - aC_{af}}{I_z u} & \frac{aC_{af} - bC_{ar}}{I_z} \end{bmatrix} \begin{bmatrix} y \\ \psi - \psi_d \\ r \end{bmatrix} + \begin{bmatrix} \frac{C_{af}}{m} \\ \frac{C_{ar}}{m} \\ \frac{aC_{af} - bC_{ar}}{I_z} \end{bmatrix} \delta_f + \begin{bmatrix} 0 \\ -u \\ -1 \end{bmatrix} r_d$$

Fig. 2. State space representation of Bicycle Model.

formulation assumes that tire slip angles remain within the linear operating region, which is reasonable for lane-following and moderately aggressive maneuvers where the controller is expected to maintain stability.

In Fig. 2, the lateral displacement is denoted by y , while the vehicle's yaw angle relative to the road is represented by ψ , r denotes the vehicle yaw rate. Regarding road geometry, ψ_d represents the road yaw angle and r_d indicates the road yaw rate. C_{af} , C_{ar} represents the front and rear cornering stiffness, I_z is the moment of inertia of the vehicle on the z axis, and m is the vehicle mass. Additionally, the longitudinal vehicle velocity is assumed to remain constant, allowing the lateral dynamics to be expressed in a linear time-invariant state-space form. These assumptions enable the development and analysis of the adaptive control law while preserving the key dynamic behaviors relevant to lateral control performance.

IV. DESIGN APPROACH

The plant transfer function is derived with MATLAB from the state space model in Fig. 2.

$$\frac{C_{af}I_z u^2 s^2 + C_{af}C_{ar}bu(a+b)s + C_{af}C_{ar}u^2(a+b)}{s^2 \left(I_z mu^2 s^2 + u [(C_{af} + C_{ar})I_z + (C_{af}a^2 + C_{ar}b^2)m] s \right) + C_{af}C_{ar}(a+b)^2 + mu^2(C_{ar}b - C_{af}a)}$$

The reference model $W_m(s)$ dictates the desired closed-loop behavior. With a desired Transfer Function:

$$W_m(s) = \frac{k_m Z_m(s)}{P_m(s)} = \frac{9}{(s+3)^2} = \frac{9}{s^2 + 6s + 9}$$

The reference model is chosen so that a relatively short time constant (≈ 1.2 s) is reached. The model is chosen to be second order to match the relative degree of the plant model.

Fixed Model Reference Control (MRC) Baseline Design:

The fixed MRC (non-adaptive) controller is derived by solving the Diophantine Equation for the nominal plant, which yields the ideal parameter vector θ^* . This vector is used as the initial condition (θ_{init}) for the MRAC system and the constant parameter vector for the Fixed MRC system.

The nominal plant $W_p(s)$ is 4th-order with 2 zeros, resulting in a Relative Degree $n^* = 2$. Controller Structure (Implicit): $u = \theta^{*T} \omega$.

Diophantine Equation: Solved in the code to find the ideal parameters θ^* and c_0^* :

$$k_p Z_p(s) \Lambda(s) W_m(s) = P_m(s) \Lambda(s) - P_p(s) \mathbf{R}(s) + k_p Z_p(s) \mathbf{S}(s)$$

where $\mathbf{R}(s)$ and $\mathbf{S}(s)$ are controller polynomials, and $\Lambda(s)$ is a stable auxiliary polynomial (chosen as $(s+3)^3$ in this case).

Limitation: The Fixed MRC performs optimally only for the nominal plant. When the dynamics change at $t = t_{change} = 8$ s, its performance degrades significantly (seen in tracking performance).

Model Reference Adaptive Control (MRAC) Design :

The MRAC maintains the same control structure as the MRC but replaces the fixed parameters θ^* with adaptive parameters $\theta(t)$.

Control Law (Adaptive):

$$u(t) = \theta^T(t) \omega(t) - \frac{e_1 \cdot \text{sgn}(c_0^*) \cdot (\phi^T \Gamma \phi)}{1 + \phi^T \phi}$$

$\theta(t)$: The vector of adaptive controller parameters. $\omega(t)$: The regressor vector, containing filtered states and signals. $e_1 = y_p - y_m$: The tracking error. ϕ : The filtered regressor vector, used for normalization and stability. The second term is the Normalization Term which ensures the system stability (SPR condition) when the input is not persistently exciting, which is critical for $n^* > 1$ designs.

Adaptive Law (Parameter Update): The parameters $\theta(t)$ are updated using the normalized σ -modification (although the code implements the basic gradient law with normalization):

$$\dot{\theta} = -\Gamma \frac{e_1 \cdot \phi \cdot \text{sgn}(c_0^*)}{1 + \phi^T \phi}$$

Γ : The Adaptation Gain Matrix (a positive definite diagonal matrix, set to $\Gamma = 20$ in the MRAC simulation).

Implementation Details and Regressor Generation: Since the relative degree is $n^* = 2$, the regressor vector ω is a mix of filtered plant input and output, the plant output itself, and the reference signal.

Regressor Vector: $\omega = [\omega_1^T, \omega_2^T, y_p, r]^T$

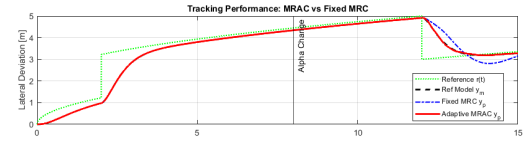


Fig. 3. Tracking Performance

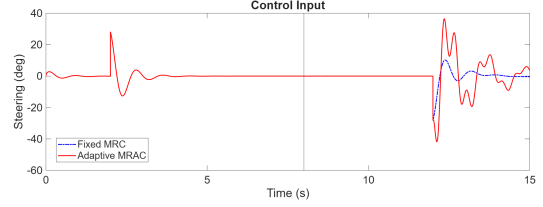


Fig. 4. Control Input

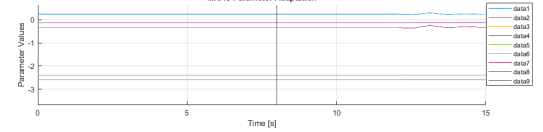


Fig. 5. Parameter Adaptation

ω_1 : Filtered input u_p (e.g., $\frac{1}{\Lambda(s)} u_p$); ω_2 : Filtered output y_p (e.g., $\frac{1}{\Lambda(s)} y_p$); y_p : Plant output; r : Reference signal; Filtered Regressor (ϕ): Required for the $n^* = 2$ design with normalization. It is generated using a stable filter with a pole at p_0 :

$$\dot{\phi} = -p_0 \phi + \omega$$

p_0 : Pole of the inner reference model, chosen from $P_m(s)$.

V. RESULTS AND ANALYSIS

Fixed MRC Performance: The tracking error ($|y_p - y_m|$) is small initially but diverges significantly after $t = 8.0$ seconds (when C_f, C_r drops by half) demonstrating its inability to handle the dynamic change.

MRAC Performance: The MRAC system exhibits a transient tracking error immediately after $t = 8.0$ seconds, but the adaptation law quickly adjusts the parameters $\theta(t)$, bringing the plant output y_p back to track y_m .

Parameter Adaptation: The parameter plots show the components of $\theta(t)$ changing dramatically at $t = 8.0$ seconds, converging to the new optimal values corresponding to the reduced cornering stiffness.

Control Input: The control input $u(t)$ (steering angle) of the MRAC controller increases and exceeds the effort applied by the Fixed MRC after $t = 8.0$, maintaining close tracking of the reference signal.

VI. CONCLUSION

This project focuses on the lateral control of vehicles under road parameter uncertainties using Model Reference Adaptive Control. Specifically, the control of a linearized bicycle model under varying cornering stiffness is analyzed. The tracking performance of the proposed MRAC controller and a baseline MRC controller is compared in a double lane change scenario. Results show that while the MRC

controller closely tracks the reference model with the correct road parameters, its performance significantly degrades after the parameter change. In comparison, the MRAC controller showed consistent tracking performance before and after the parameter change. Therefore, the proposed MRAC controller enhances safety during emergency vehicle lateral control under uncertain cornering stiffness.

REFERENCES

- [1] A. G. Ulsoy, H. Peng, and M. Çakmakci, *Automotive Control Systems*. Cambridge University Press, 2012.

VII. RESPONSES TO PEER FEEDBACK

Additional Results

Different Reference Signal:

Based on presentation feedback, the tracking performance of the MRAC controller under sine input, increased track curvature, and increased lateral step are further presented in Fig. 6, 7, 8.

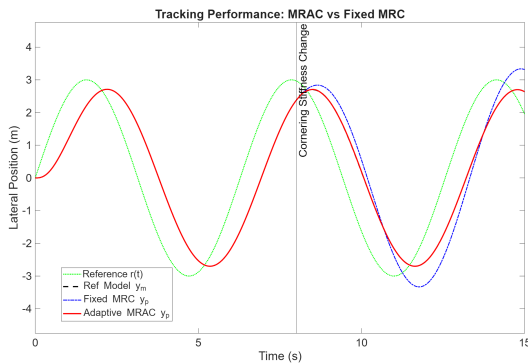


Fig. 6. Tracking performance under sine input.

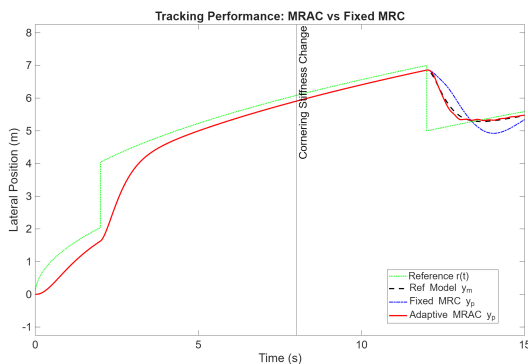


Fig. 7. Tracking performance on a higher curvature road.

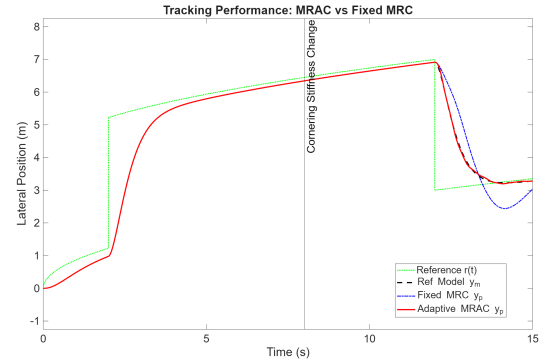


Fig. 8. Tracking performance for a larger lateral displacement reference.

Control Saturation:

Effect of a saturating control input is shown in Fig. 9. Note that Limit 100° represents the original controller performance.

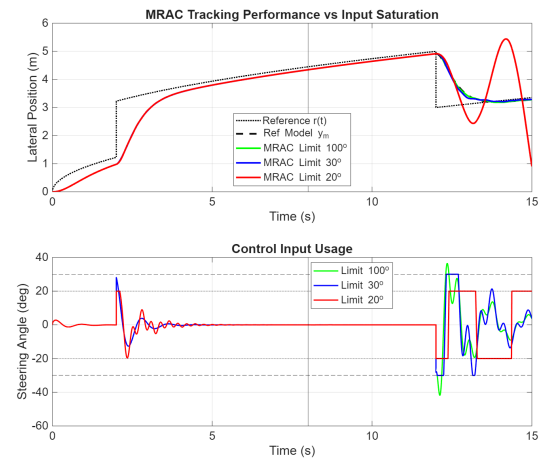


Fig. 9. Tracking performance under input saturation.

Different Longitudinal Velocities:

The tracking performance of the controller under different longitudinal velocities is shown in Fig. 10.

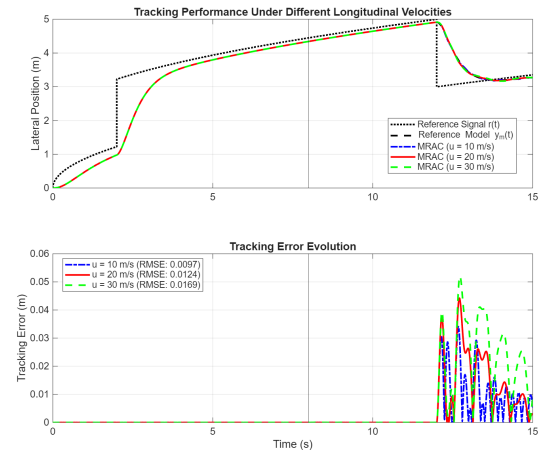


Fig. 10. Tracking performance under different longitudinal velocities.

Feedback 1

Comment 1: I believe communication skills can be improved by speaking a little bit louder.

Response: Thanks for the advice! We will definitely work on speaking louder.

Feedback 2

Comment 1: More results with different tunings would be very interesting to see.

Response: We did try to add control input saturation, tune the regressor to improve the oscillations in Control input.

Comment 2: How was the state space model derived? about what operating point?

Response: For the details of the derivation, please see A. G. Ulsoy, H. Peng, and M. Çakmakci, *Automotive Control Systems*. Cambridge: Cambridge University Press, 2012.

Comment 3: even more scenarios and examples would be great to see, to further differentiate the adaptive vs. non adaptive methods

Response: From Fig. 6, where a sine wave is used as the reference signal, the baseline MRC deviates from the reference model as soon as the road parameter changes, while the MRAC controller closely tracks the reference.

Feedback 3

Comment 1: have you considered how your MRAC controller would perform at different speed and whether a speed-dependent reference model might be needed?

Response: In Fig. 10, we further analyzed the MRAC performance at different speeds, the performance remains superior to the Baseline MRC performance. However, the tracking error slightly increases as the speed increases.

Feedback 4

Comment 1: Try clamping control inputs and evaluate its performance. The steering rate for MRAC presented might not be practical.

Response: We tried clamping the steering angle to $\pm 30^\circ$, the MRAC controller maintained good performance with some degradation. However, if the steering angle is clamped to $\pm 20^\circ$, the controller will diverge.

Comment 2: Maybe introduce actuator limits as a feedforward term?

Response: This is a great direction, but with the current time frame we are not able to test this approach.

Feedback 5

Comment 1: Some variation in the emergency maneuver could be interesting to show, validating your model for different maneuvers. Different road curvature, length of maneuver, amount of required lateral movement etc.

Response: We did some further experiments on different reference signals in Fig. 6, 7, 8. The MRAC controller maintained good performance under these scenarios.

Feedback 6

Comment 1: However, my question is when the driver panics in losing traction condition most of them might tend to over steering to an direction very fast. Could the controller handle the fast sudden change? Maybe limitations on controller is needed for saturation?

Response: The controller will be able to handle sudden changes in the reference signal even though it might not be able to perfectly track the sudden changes. The MRAC Controller is derived to mimic the reference model's response to the reference signal. And the reference signal used in our scenario includes a sudden step of reference signal, and the controller tracks the reference model pretty well.

In terms of the controller saturation, Fig. 9 shows the controller's performance with different saturation levels implemented. Results show that the controller will not be able to track the reference under a tight saturation level. Some additional safety fallback will need to be added if this were to be implemented in practice.

Feedback 7

Comment 1: If you have time, something that I think would really take your report to the next level is trying different kinds of evasive action inputs. Sine with dwell, fishhook, or a moose test could all be interesting to test your controller against.

Response: We further added a sine input response in Fig. 6. The original double lane change maneuver can be considered as a variant of the moose test as it requests a sudden change in lateral position.

Feedback 8

Comment 1: Why is there a large control input in MRAC compared to MRC in the regime where the stiffness changes? Is it purely cause of parameter estimation effort?

Response: Part of the increase in MRAC control effort may be attributed to the estimation effort, where the controller picks up minor oscillations in the feedback signal and amplifies it into a larger control effort. Another reason is that after the reduce in the cornering stiffness, a larger steering angle is required for the same amount of lateral force, hence a larger control effort is expected.

Feedback 9

Comment 1: How did you choose the reference model?

Response: We went through a few books and other literature, this reference model stands out as it is only fourth order (others have 5, 6) and relative degree 2 (simple enough for Lyapunov MRAC implementation).

Comment 2: Are you concerned with the steering degree angle? 40 degrees seems very high. Do you think you could implement control limits?

Response: Yes, we tried to implement control limits on the steering angle. The performance maintained pretty well with a limit of $\pm 30^\circ$, while the controller diverges when we further tightened it to $\pm 20^\circ$, this is certainly a limit of our

approach, where we cannot impose constraints on the control input.

Comment 3: I don't understand the current reference profile, is that representing a car skidding to the side?

Response: The reference signal is representing a double lane change maneuver, where the car switches lane and then switches back, on a slightly curved road, that is why the lateral deviation is slightly increasing.

Feedback 10

Comment 1: The technical work was solid, but the presentation would improve with clearer speaking and more confidence from the presenters. When confidence is low, it can be harder for the audience to trust or fully understand the results.

Response: Thanks for the advice. We will try to improve this in future presentations.

Comment 2: It would also help to better explain how the reference model was chosen, especially why that specific transfer function was used. This would make the control design easier to follow.

Response: Thanks for the advice. The reference model was chosen to mimic a ideal tracking performance with a time constant of about 1.2 s. The transfer function $\frac{9}{(s+3)^2}$ was chosen for simplicity given the time constant constraint.

Feedback 11

Comment 1: Maybe use optimal methods as well(LQR and stuff)?

Response: Didn't get time to work on pure LQR optimization but did try to implement the LQR based MRAC control design.

VIII. ATTACHMENT

```
clear; clc; close all;

% 1. Simulation & Vehicle Parameters
t_change = 8.0; % Time at which cornering stiffness drops by 50% [s]

m = 1500; % Mass [kg]
Iz = 2500; % Yaw Moment of Inertia [kg m^2]
Ca_f = 55000; % Nominal Front Cornering Stiffness [N/rad]
Ca_r = 60000; % Nominal Rear Cornering Stiffness [N/rad]
a = 1.2; % CG to Front Axle [m]
b = 1.4; % CG to Rear Axle [m]
u = 20; % Longitudinal Velocity [m/s]

% Pack vehicle parameters for simulation
vp.m = m; vp.Iz = Iz; vp.Ca_f = Ca_f; vp.Ca_r = Ca_r;
vp.a = a; vp.b = b; vp.u = u; vp.t_change = t_change;

% 2. Nominal Plant Model Setup (For Initialization)
% States: x = [y; y_dot; psi; r]
% Input: delta_f
% Output: y

% A matrix (Nominal)
a22 = -(Ca_f + Ca_r) / (m * u);
a23 = (Ca_f + Ca_r) / m;
a24 = (b * Ca_r - a * Ca_f) / (m * u);
a42 = (b * Ca_r - a * Ca_f) / (Iz * u);
a43 = (a * Ca_f - b * Ca_r) / Iz;
a44 = -(a^2 * Ca_f + b^2 * Ca_r) / (Iz * u);

b2 = Ca_f / m;
b4 = a * Ca_f / Iz;
```

```
A_p = [0, 1, 0, 0;
        0, a22, a23, a24;
        0, 0, 0, 1;
        0, a42, a43, a44];

B_p = [0; b2; 0; b4];
C_p = [1, 0, 0, 0];
D_p = 0;

n = 4; % Plant Order

% Transfer Function
[num_p, den_p] = ss2tf(A_p, B_p, C_p, D_p);
num_p(abs(num_p) < 1e-10) = 0;
den_p(abs(den_p) < 1e-10) = 0;

first_nc_idx = find(abs(num_p) > 1e-10, 1);
k_p = num_p(first_nc_idx);
Z_p = num_p(first_nc_idx:end) / k_p;
P_p = den_p;

% 3. Reference Model Setup
% W(s) = 1 / (s + 3)^2
Z_m = 1;
P_m = [1, 6, 9];
K_m = P_m(3);
p0 = P_m(2)/2; % Pole for n*=2 design (Wm = H(s)/(s+p0) -> H(s) must be SPM)

% 4. HRC Initialization (Solving for Theta*)
% Filter Lambda(s) = (s+10)^3 = s^3 + 30s^2 + 300s + 1000
Lambda = poly([-3, -3, -3]);

% Diophantine Equation Solution
c_0_star = k_m / k_p;

RHS_poly = conv(conv(Z_p, Lambda), P_m) - conv(Lambda, P_p);
RHS_vec = [zeros(1, 0 - length(RHS_poly)), RHS_poly]';

M = zeros(8, 7);
term_t1_1 = - [P_p, 0, 0];
term_t1_2 = - [0, P_p, 0];
term_t1_3 = - [0, 0, P_p];
M(i, 1) = [zeros(8-length(term_t1_1),1); term_t1_1'];
M(i, 2) = [zeros(8-length(term_t1_2),1); term_t1_2'];
M(i, 3) = [zeros(8-length(term_t1_3),1); term_t1_3'];

KZp = k_p * Z_p;
term_t2_1 = - conv(KZp, [1, 0, 0]);
term_t2_2 = - conv(KZp, [1, 0]);
term_t2_3 = - KZp;
M(i, 4) = [zeros(8-length(term_t2_1),1); term_t2_1'];
M(i, 5) = [zeros(8-length(term_t2_2),1); term_t2_2'];
M(i, 6) = [zeros(8-length(term_t2_3),1); term_t2_3'];

term_t3 = - conv(KZp, Lambda);
M(i, 7) = [zeros(8-length(term_t3),1); term_t3'];

mrc_params = pinv(M) * RHS_vec;

theta_1_star = mrc_params(1:3);
theta_2_star = mrc_params(4:6);
theta_3_star = mrc_params(7);

% Construct Initial Parameter Vector Theta_0
% Order in code: [theta_1; theta_2; theta_3; c_0]
theta_init = [theta_1_star; theta_2_star; theta_3_star; c_0_star];
```

```
% 5. Simulation Setup
% Filter Matrices (F, g) for alpha(s)/Lambda(s)
l2 = Lambda(2); l1 = Lambda(3); l0 = Lambda(4);
F = [-l2, -l1, -l0;
      1, 0, 0;
      0, 1, 0];
g = [1; 0; 0];

% Reference Model State Space
[A_m, B_m, C_m, D_m] = tf2ss(K_m*Z_m, P_m);

% Simulation Time
T_sim = 15;
t_span = [0 T_sim];

% Initial Conditions (Common for both)
X0 = zeros(28, 1);
X0(21:28) = theta_init; % Initialize parameters

% Options
options = odeset('RelTol', 1e-6, 'AbsTol', 1e-6);

% --- SIMULATION 1: MRAC (Adapting) ---
% Gamma_mrac = 200 * eye(8);
% Gamma_mrac = 20 * eye(8);
[t_mrac, X_mrac] = ode45(@(t, x) sys_dynamics(t, x, vp, F, g, A_m, B_m, C_m, D_m, ...
        Gamma_mrac, p0, sign(c_0_star)), t_span, X0, options);

% --- SIMULATION 2: Fixed HRC (Non-adapting) ---
% We simply set Gamma to zero. This keeps theta constant at theta_init.
Gamma_mrac = zeros(8);
[t_mrac, X_mrac] = ode45(@(t, x) sys_dynamics(t, x, vp, F, g, A_m, B_m, C_m, D_m, ...
        Gamma_mrac, p0, sign(c_0_star)), t_span, X0, options);

% 6. Plotting & Analysis
% Helper function to extract signals
[y_p_mrac, u_mrac, r_mrac, y_mrac] = process_results('mrac', X_mrac, Ca_f, D_m, Gamma_mrac, c_0_star);
[y_p_mrac, u_mrac, r_mrac, y_mrac] = process_results('mrac', X_mrac, Ca_f, D_m, Gamma_mrac, c_0_star);
figure('color','b', 'Position', [100 100 1000 800]);
subplot(3,1,1);
plot(t_mrac, r_mrac, 'b-', 'LineWidth', 1.5); hold on;
plot(t_mrac, y_p_mrac, 'b-', 'LineWidth', 2);
plot(t_mrac, y_p_mrac, 'b-', 'LineWidth', 1.5);
plot(t_mrac, y_p_mrac, 'b-', 'LineWidth', 2);
xlabel('Time [s]');
ylabel('Lateral Deviation [m]');
title('Tracking Performance: MRAC vs Fixed HRC');

subplot(3,1,2);
plot(t_mrac, rad2deg(u_mrac), 'b-', 'LineWidth', 1.5); hold on;
plot(t_mrac, rad2deg(u_mrac), 'b-', 'LineWidth', 1.5);
xlabel('Time [s]');
ylabel('Steering [deg]');
title('Control Input: Steering vs Time');

subplot(3,1,3);
% Plotting only MRAC parameters (HRC are constant)
theta_init = X_mrac(1, 21:28);
hold on;
for i = 1:8
    plot(t_mrac, theta_init(i,:), 'b');
end
xlabel('Time [s]');
ylabel('Parameter Values');
```

```

function [y_p, u_control, r_trace, y_m] = process_results(T, X, C_m, D_m, gamma, c0_star)
    y_p = X(:, 1);
    y_m = zeros(length(T), 1);
    u_control = zeros(length(T), 1);
    r_trace = zeros(length(T), 1);

    for i = 1:length(T)
        t_curr = T(i);
        r_val = reference_signal(t_curr);
        r_trace(i) = r_val;

        % Ref Model Output
        x_m_curr = X(i, 11:12);
        y_m(i) = C_m * x_m_curr + D_m * r_val;

        % Reconstruct Control
        omega_1 = X(i, 5:7);
        omega_2 = X(i, 8:10);
        phi = X(i, 13:20);
        theta = X(i, 21:28);

        omega = [omega_1; omega_2; y_p(i); r_val];
        e1 = y_p(i) - y_m(i);
        norm_factor = 1 + phi' * phi;

        correction_term = (e1 * sign(c0_star) * (phi' * Gamma * phi)) / norm_factor;
        u_val = theta' * omega - correction_term;

        % No Saturation here!
        u_control(i) = u_val;
    end
end

```

```

function dx = sys_dynamics(t, X, vp, F, g, A_p, B_m, C_m, D_m, ...
    Gamma, p0, sgn_c0)

```

```

    r_cmd = reference_signal(t);

    % --- PLANT DYNAMICS UPDATE (Based on time t) ---
    m = vp.m; u = vp.u; Iz = vp.Iz; a = vp.a; b = vp.b;

    if t >= vp.t_change
        % Reduced stiffness (e.g. icy patch)
        Caf_cur = vp.Ca_f * 0.5;
        Car_cur = vp.Ca_r * 0.5;
    else
        % Nominal stiffness
        Caf_cur = vp.Ca_f;
        Car_cur = vp.Ca_r;
    end

    % Recompute Plant Matrices for current time step
    a22 = -(Caf_cur + Car_cur) / (m * u);
    a23 = (Caf_cur + Car_cur) / m;
    a24 = (b * Car_cur - a * Caf_cur) / (m * u);
    a42 = (b * Car_cur - a * Caf_cur) / (Iz * u);
    a43 = (a * Caf_cur - b * Car_cur) / Iz;
    a44 = -(a^2 * Caf_cur + b^2 * Car_cur) / (Iz * u);
    b2 = Caf_cur / m;
    b4 = a * Caf_cur / Iz;

    A_p_cur = [0, 1, 0, 0;
                0, a22, a23, a24;
                0, 0, 0, 1;
                0, a42, a43, a44];
    B_p_cur = [0; b2; 0; b4];
    % -----
    I

```

```

x_p = X(1:4);
omega_1 = X(5:7);
omega_2 = X(8:10);
x_m = X(11:12);
phi = X(13:20);
theta = X(21:28); % [theta1(3); theta2(3); theta3(1); c0(1)]

y_p = X_p(1);

% Reference Model Output
y_m = C_m * x_m + D_m * r_cmd;

% 1. Construct Regressor Omega
% omega = [omega_1; omega_2; y_p; r]
omega = [omega_1; omega_2; y_p; r_cmd];

% 2. Error Calculation
e1 = y_p - y_m;

% 3. Control Law Calculation (n* = 2) with Normalization
norm_factor = 1 + phi' * phi;

% u_p = theta' * omega - (e1 * sgn(c0) * phi' * Gamma * phi) / norm_factor
correction_term = (e1 * sgn_c0 * (phi' * Gamma * phi)) / norm_factor;
u_raw = theta' * omega - correction_term;

% No Saturation here!
u_p = u_raw;

% 4. Adaptation Law with Normalization
theta_dot = (-Gamma * e1 * phi * sgn_c0) / norm_factor;

```

```

% 5. State Derivatives

% Plant (Using updated A_p_cur and B_p_cur)
dx_p = A_p_cur * x_p + B_p_cur * u_p;

% Filters (omega generation)
domega_1 = F * omega_1 + g * u_p;
domega_2 = F * omega_2 + g * y_p;

% Reference Model
dx_m = A_m * x_m + B_m * r_cmd;

% Filter for phi (n* = 2 specific)
dphi = -p0 * phi + omega;

dx = [dx_p; domega_1; domega_2; dx_m; dphi; theta_dot];
end

function r = reference_signal(t)
    if t >= 2 && t < 12
        r = 2 + 3 * sqrt(t/12);
    else
        r = 0 + 3 * sqrt(t/12);
    end
end

```